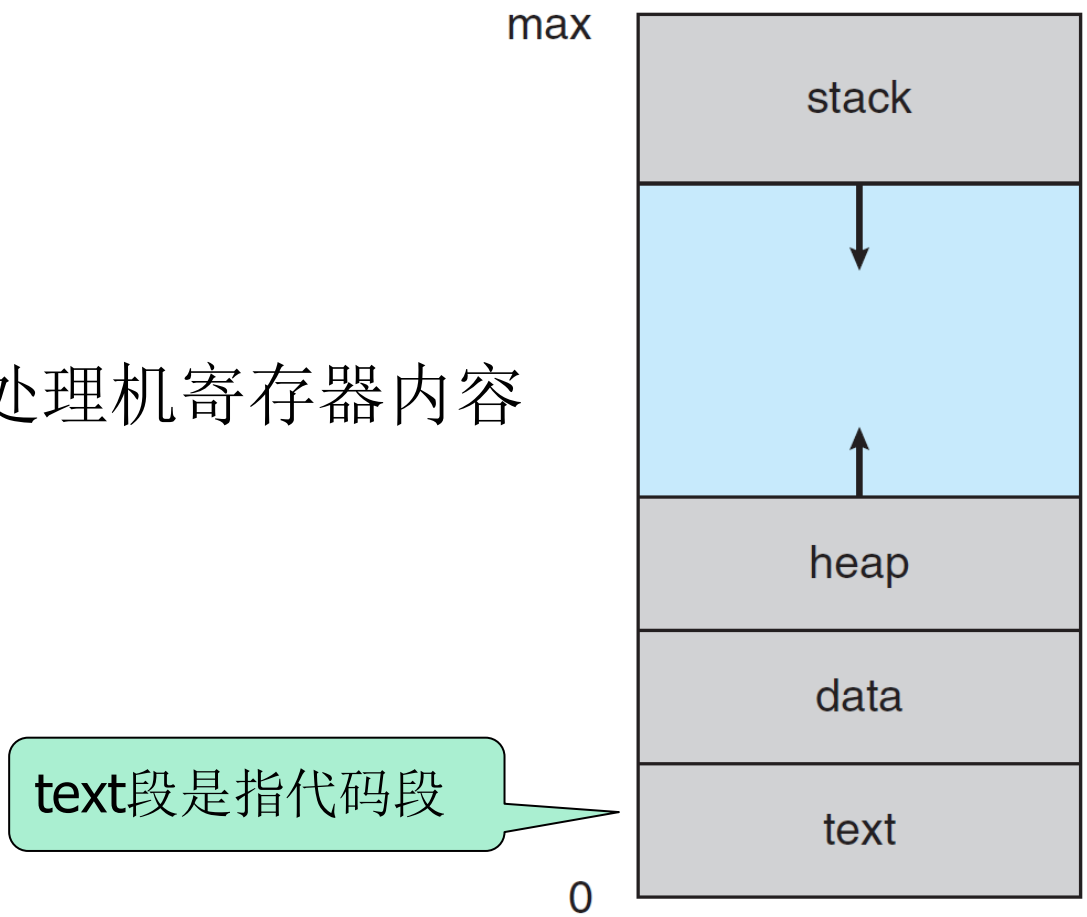
第3章 进程

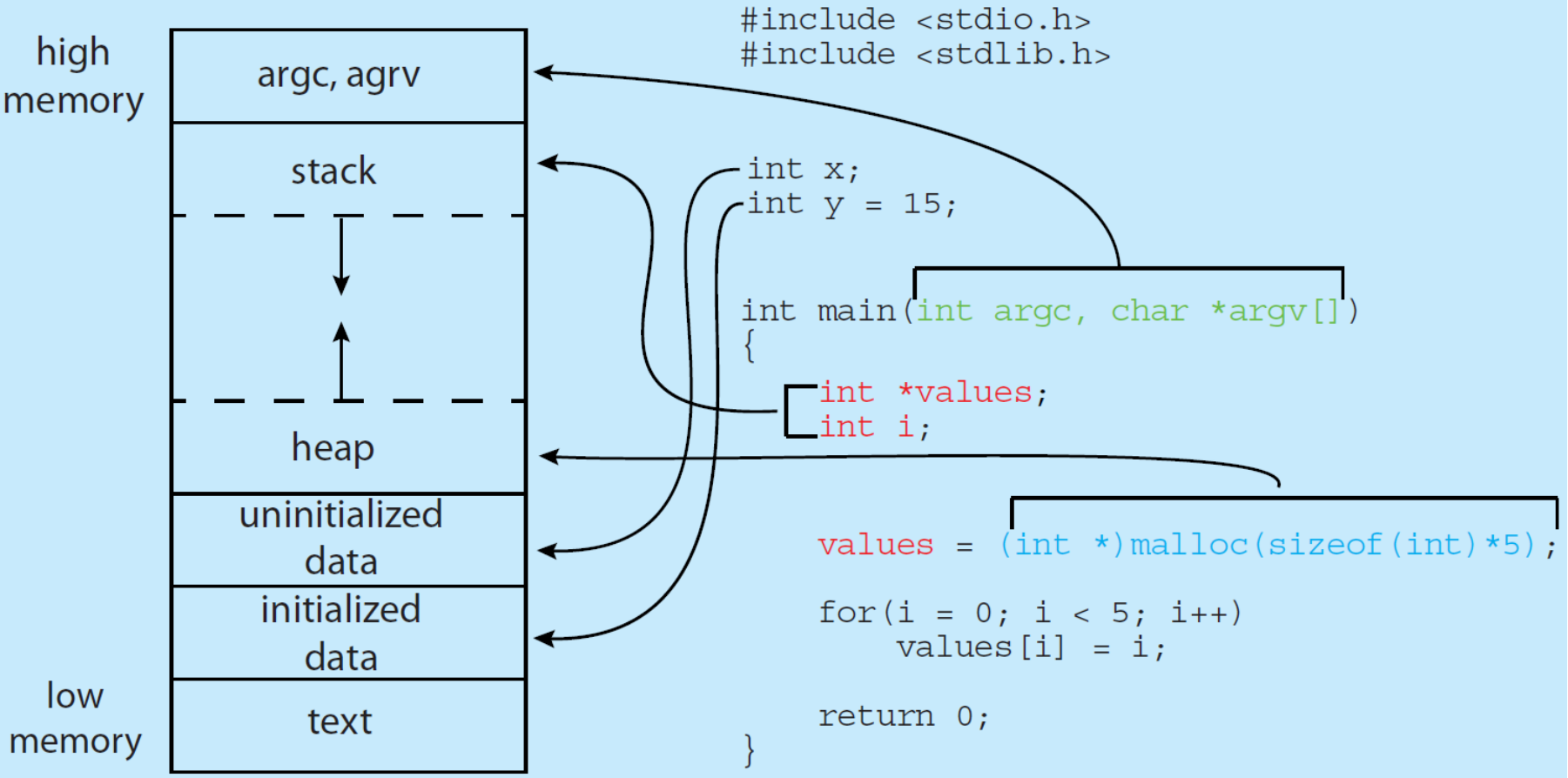
- 本章内容
- 1. 进程的概念
- 2. 进程调度
- 3. 进程操作
- 4. 进程间通信

进程

- 进程是执行中的程序，这是一种非正式的说法
- 一个进程包括
  - 代码段
  - 程序计数器和处理机寄存器内容
  - 栈
  - 数据段



内存中的进程


high memory
argc, agrv
stack
heap
uninitialized data
initialized data
text
low memory
#include <stdio.h>
#include <stdlib.h>
int x;
int y = 15;
int main(int argc, char *argv[])
{
int *values;
int i;
values = (int *)malloc(sizeof(int)*5);
for(i = 0; i < 5; i++)
values[i] = i;
return 0;
}

进程的定义

- 进程有多种定义，下面列举一些有代表性的定义
  - 进程是程序在处理器上的一次执行过程
  - 进程是可以和别的计算并行执行的计算
  - 进程是程序在一个数据集合上运行的过程，是系统进行资源分配和调度的一个独立单位
  - 进程是一个具有一定功能的程序关于某个数据集合的一次运行活动

进程要突出“执行”，不只是程序，执行的状态通过各种数据反映出来

进程的特征

- 动态性：进程是程序的一次执行过程
  - 因创建而产生，因调度而执行，因无资源而暂停，因撤消而消亡。而程序是静态实体
- 并发性：多个进程可在一段时间内同时运行
- 独立性：进程是独立运行的基本单位
  - 在传统OS中是分配资源和调度的基本单位
- 异步性：以各自独立不可预知的速度推进
- 结构性：由程序段、数据段及进程控制块组成，又称为进程映像

传统，是指尚未引入线程，本章均针对传统OS，不涉及线程

进程与程序的关系

- 进程是动态概念，程序是静态概念
  - 进程是程序在处理机上的一次执行过程，而程序是指令的集合
- 进程是暂时的，程序是永久的
  - 进程是一个状态变化的过程；程序可长久保存
- 进程的组成包括程序、数据和进程控制块
- 一个程序可对应多个进程；一个进程可包括多个程序
- 进程可以创建新进程，程序不能形成新程序

进程状态

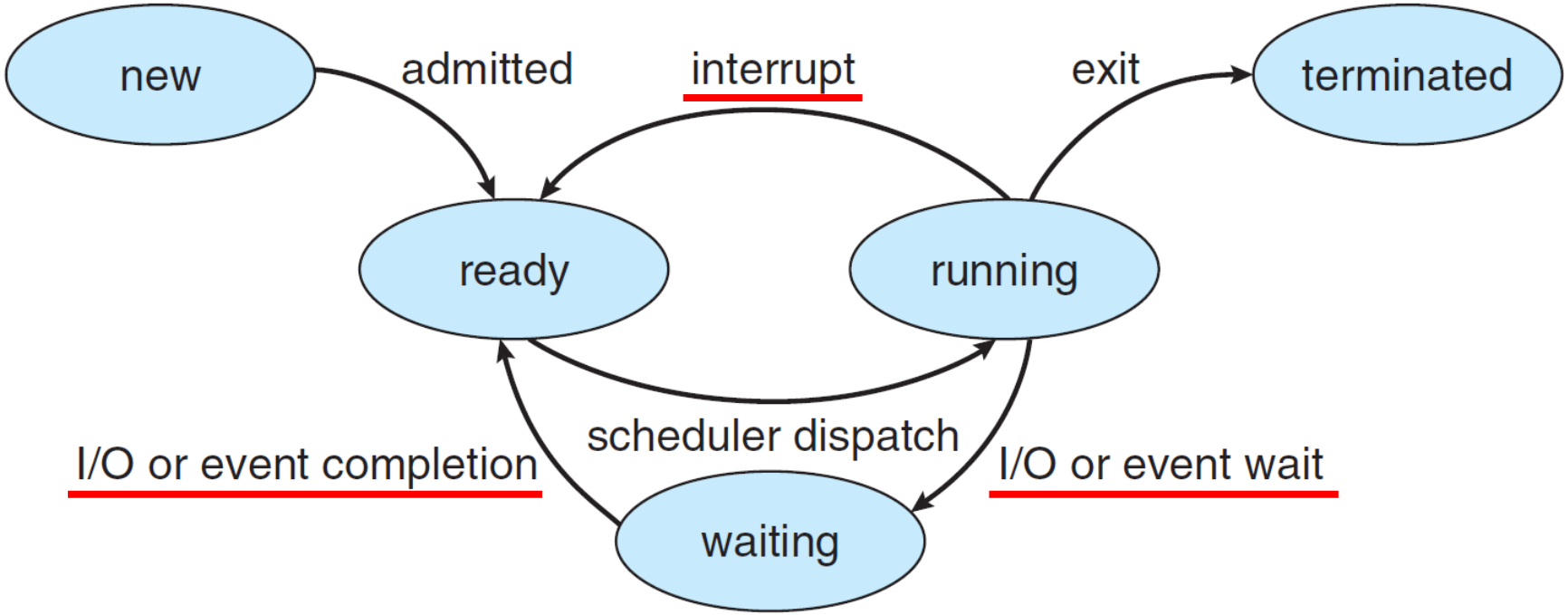
- 进程执行时会改变状态。进程状态包括：
  - 新建：刚创建，未进入就绪队列，又称创建状态
  - 运行：获得必要的资源并正在处理机上执行
  - 等待：又称阻塞、睡眠，进程由于发生某事件而暂时无法执行下去（如等待输入/输出完成），这时即使把CPU分配给该进程，它也无法运行
  - 就绪：已获得除CPU以外的所有资源，一旦分配了处理机就可以立即执行
  - 终止：正常或异常结束，系统已释放它所占用的资源，但尚未将它撤消时的状态

主动放弃CPU，流畅省电的保障

不久必然得以运行，多个就绪进程意味着CPU忙

进程状态图

- 数量
  - 运行态进程数量受限于CPU数量
  - 就绪态、阻塞态有队列



进程状态图

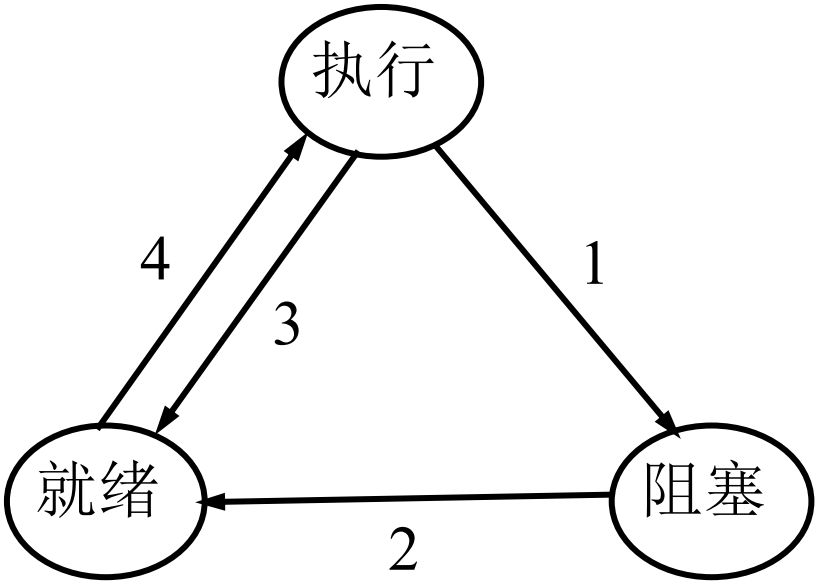
- 中断
  - 典型如时钟，也可能其它来源如鼠标，OS借机夺回CPU，趁机切换
  - 并不总引起切换，如鼠标移动，触发内核中断服务，但处理完成后决定回到原进程，不切换
- IO与等待
  - 如读写文件、等待用户操作、睡眠等待
  - 一定由程序主动发起（推进到无法前进的状态）
  - 并不总造成阻塞，如非阻塞IO、自旋等待

进程状态图

- IO与等待完成
  - 由OS感知并处理（此时进程处于阻塞态，不可能感知处理任何事情）
  - 由此可知，先前阻塞时不仅释放CPU，还在系统中登记了唤醒条件
- 思考
  - 如果系统中有N个进程，运行进程最多几个最少几个；就绪呢；阻塞呢？
  - 有没有这样的状态转换：阻塞—运行，就绪—阻塞，为什么？

因果变迁

- 如果一个进程的状态变化A的发生，会引起另一个进程的状态变化B的发生，则称A、B之间是因果变迁

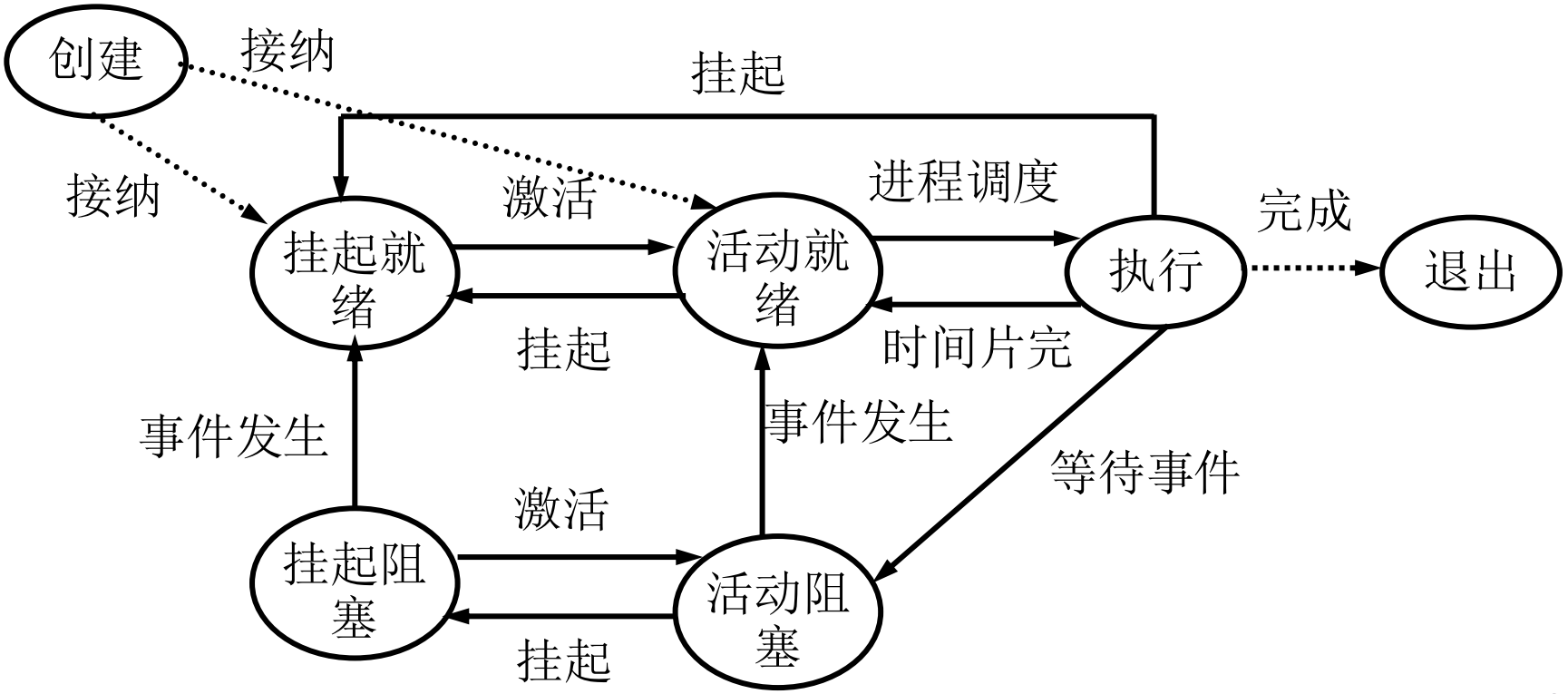


以下哪些是因果变迁
a）3->4
b）2->4
c）1->2

进程的挂起状态

- 也称静止状态，对应正常情况下的活动状态
  - 挂起是被动状态，被外界（OS或其它进程）强制剥夺执行权，不影响就绪、阻塞状态
  - 就绪、阻塞是自身健康状态，挂起、活动是人身自由状态，二者并存
- 进程挂起的原因
  - 系统故障或功能受到破坏：挂起以修复故障
  - 检查中间结果：挂起进程以便检查
  - 资源不足：挂起进程以腾出资源
  - 内存不足：在外存挂起

有挂起状态的进程状态转换图

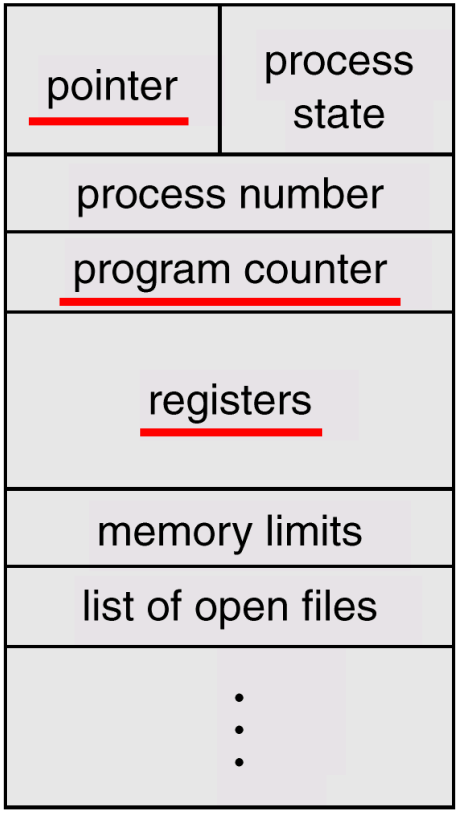


进程控制块

- 每个进程在操作系统内用进程控制块表示，Process Control Block（PCB）
  - PCB是OS内核中描述和管理进程的数据结构
  - 它是进程实体的一部分
  - 操作系统通过PCB感知进程的存在，PCB是进程存在的唯一标志
  - 类似学校管理学生的档案表，记录重要信息，不干涉日常行为

进程控制块

- 不同OS中PCB的结构不同，但通常包括下面所列出的内容



- Process state
- Program counter
- CPU Registers
- CPU-scheduling information
- Memory-management information
- Accounting information
- I/O status information

PCB包含的内容

- 进程标识符：惟一标识进程的一个标识符或整数
- 进程当前状态：说明进程当前所处状态
- 进程队列指针：用于记录PCB队列中下一个PCB的地址（用于构建链表）
- 程序和数据地址：进程的程序和数据在内存或外存中的存放地址
- 进程优先级：反映进程获得CPU的优先级别

PCB包含的内容

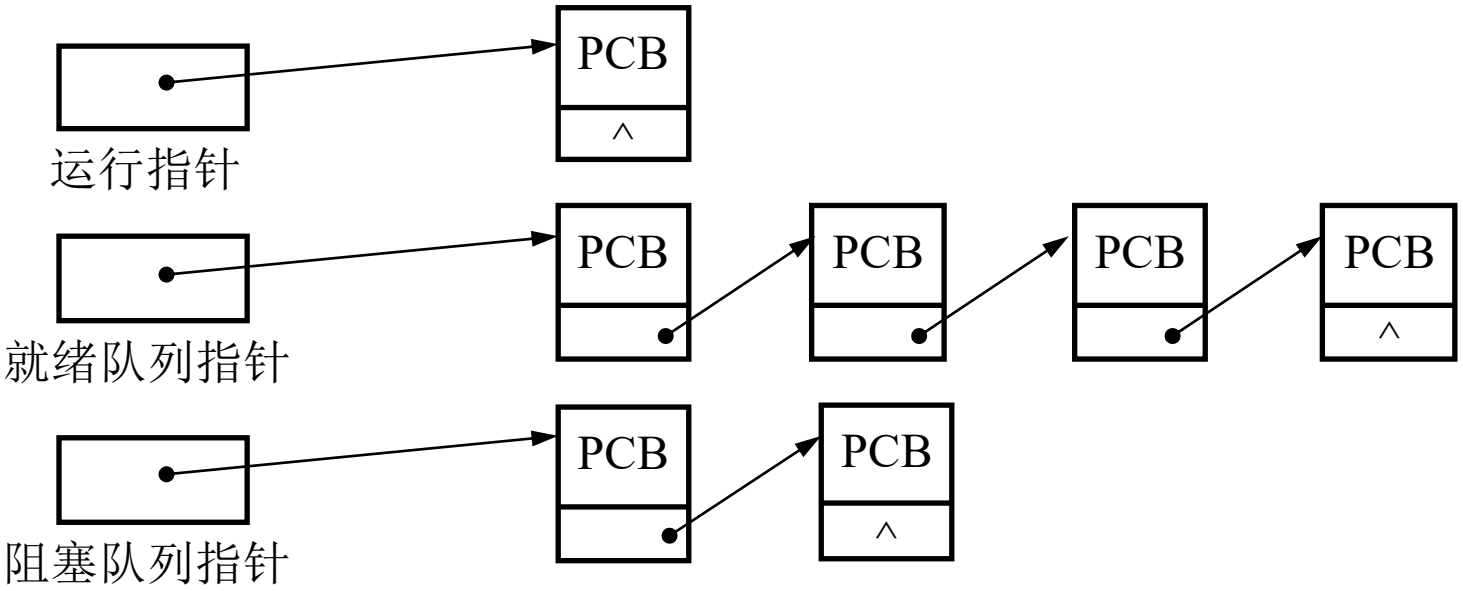
- CPU现场保护区：CPU现场信息的存放区域，包括通用寄存器、程序计数器、程序状态字等（进程失去CPU前在此保存进度）
- 通信信息：与其他进程所发生的信息交换
- 家族关系：指明本进程与家族的关系，如父子进程标识
- 资源清单：列出进程所需资源及当前已分配资源

PCB的组织

- 常用的组织方式
  - 线性方式：顺序存放在一片连续内存中

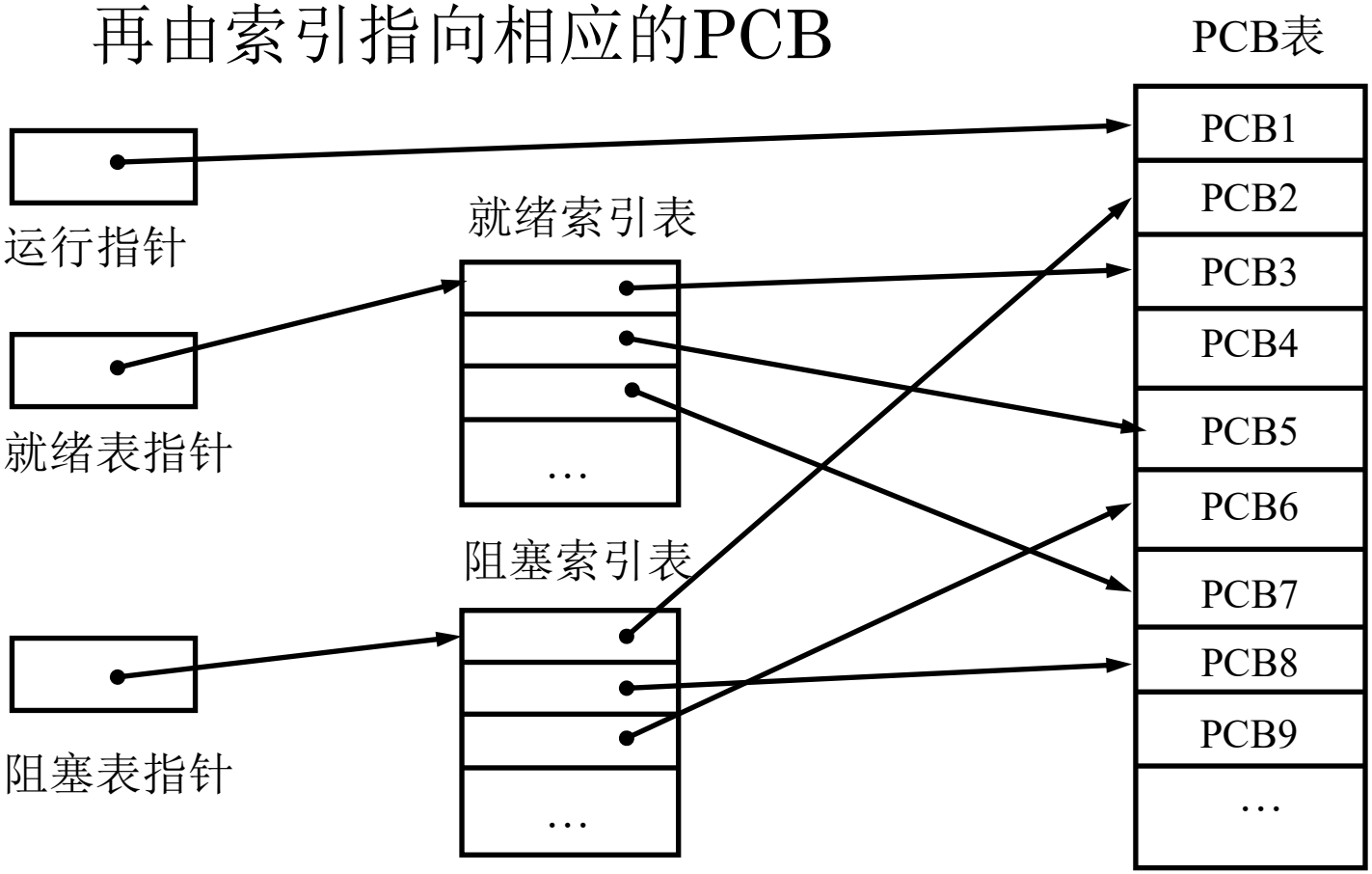
PCB_1	PCB_2	PCB_3	…	PCB_{n-2}	PCB_{n-1}	PCB_n

  - 链表方式：将同一状态的PCB组成一个链表



PCB的组织

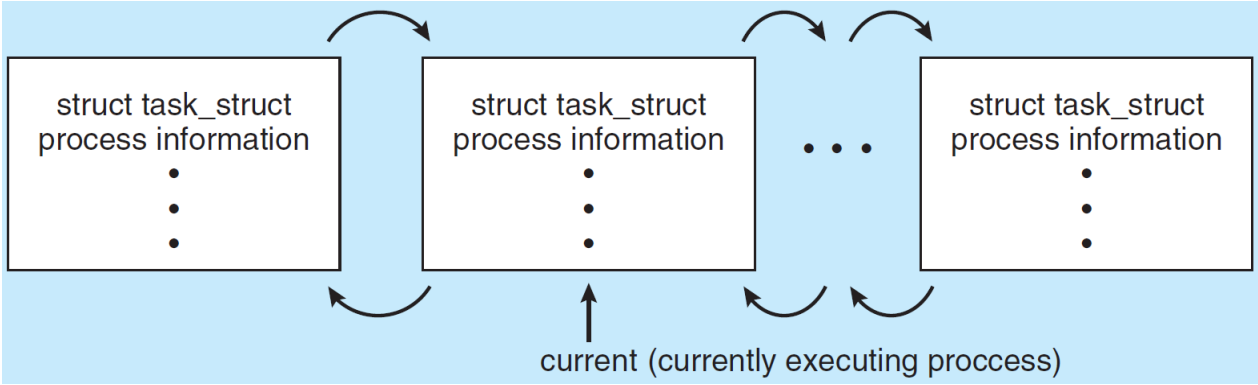
- 常用的组织方式
  - 索引方式：将同一状态的进程归入一个索引表，再由索引指向相应的PCB



Linux案例

- task_struct，位于include/linux/sched.h

```
struct task_struct {
    ......
    long state;                    /* 进程状态 */
    struct sched_entity se;        /* 调度信息 */
    struct list_head tasks;        /* 双向链表指针 */
    struct task_struct *parent;    /* 父进程 */
    struct list_head children;     /* 子进程链表 */
    struct list_head sibling;      /* 兄弟进程链表 */
    struct files_struct *files;    /* 打开的文件 */
    struct mm_struct *mm;          /* 内存管理 */
};
```

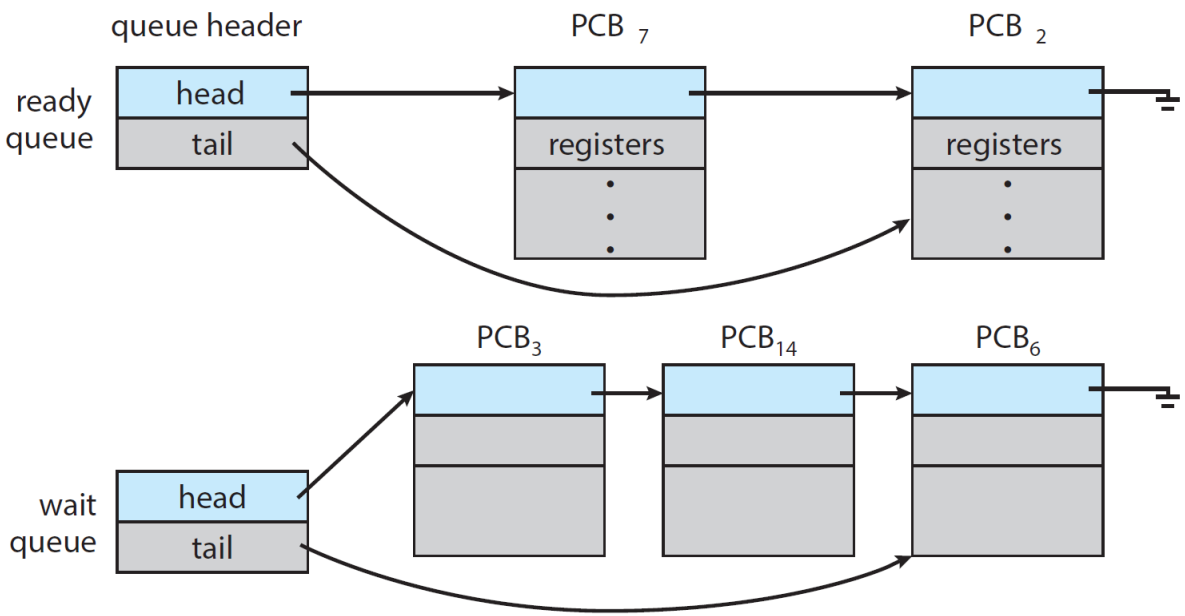


第3章 进程

- 本章内容
- 1. 进程的概念
- 2. 进程调度
- 3. 进程操作
- 4. 进程间通信

调度队列

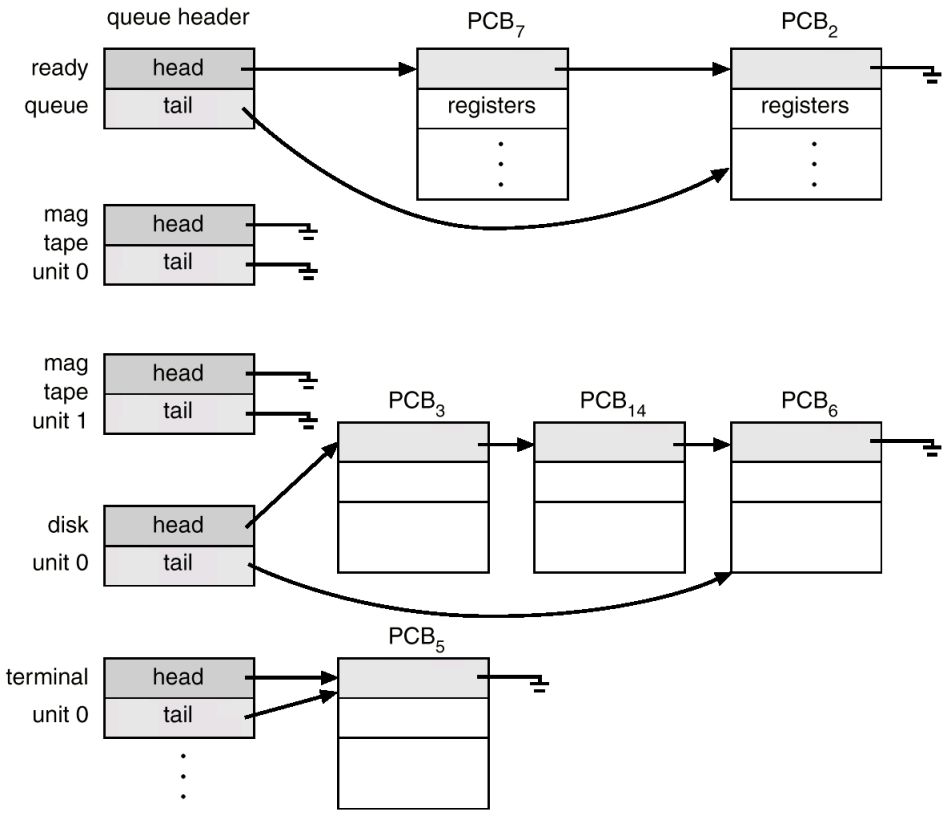
- 就绪队列：所有就绪进程的PCB集合
- 阻塞队列：阻塞态进程PCB构成的队列



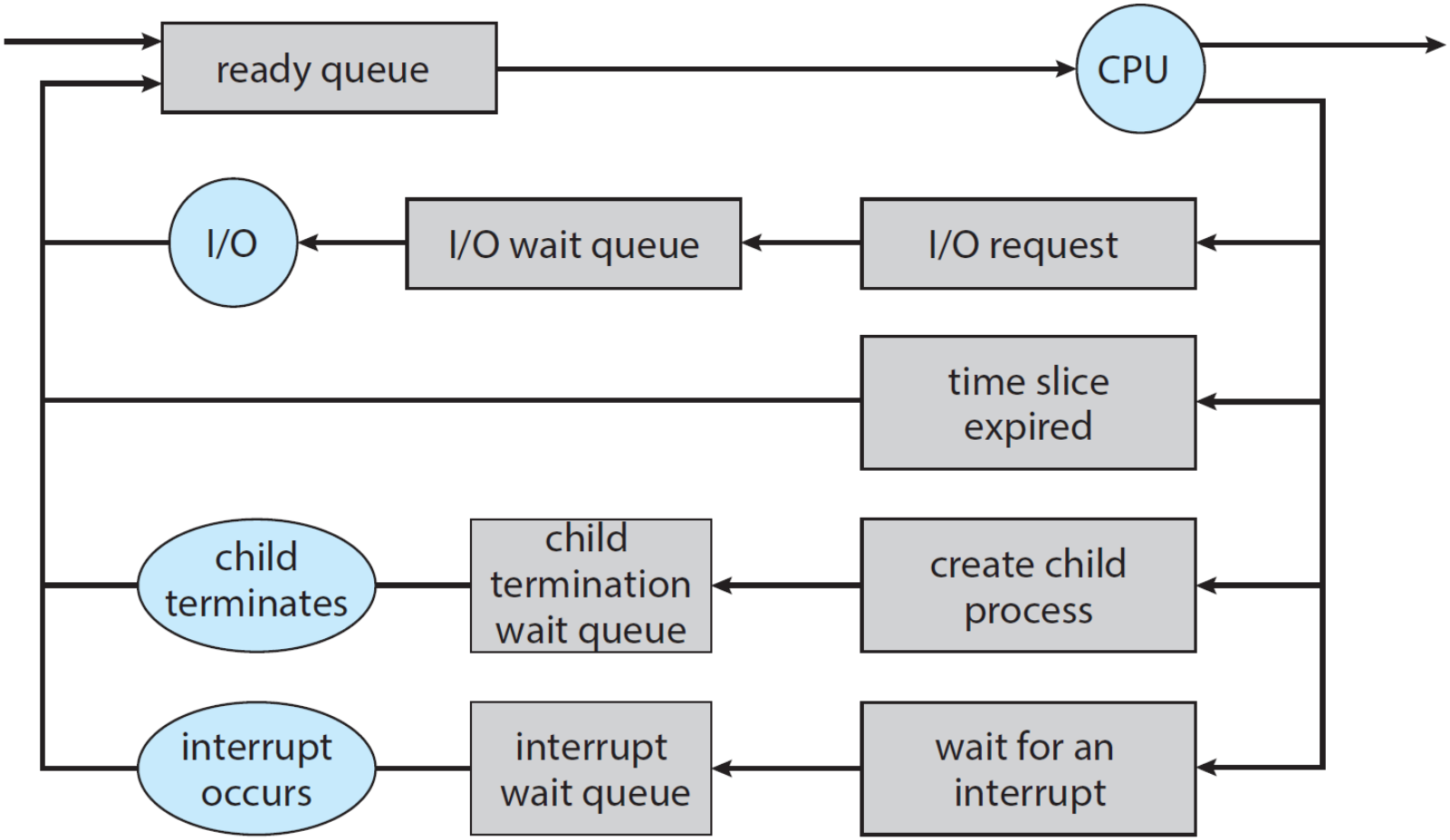
调度队列

- 思考：就绪和阻塞队列需要一个还是多个？
  - 就绪队列：可以一个，但通常每个CPU一个
  - 阻塞队列：每种阻塞原因一个以方便唤醒时查找

问题3.1：为什么？



调度队列

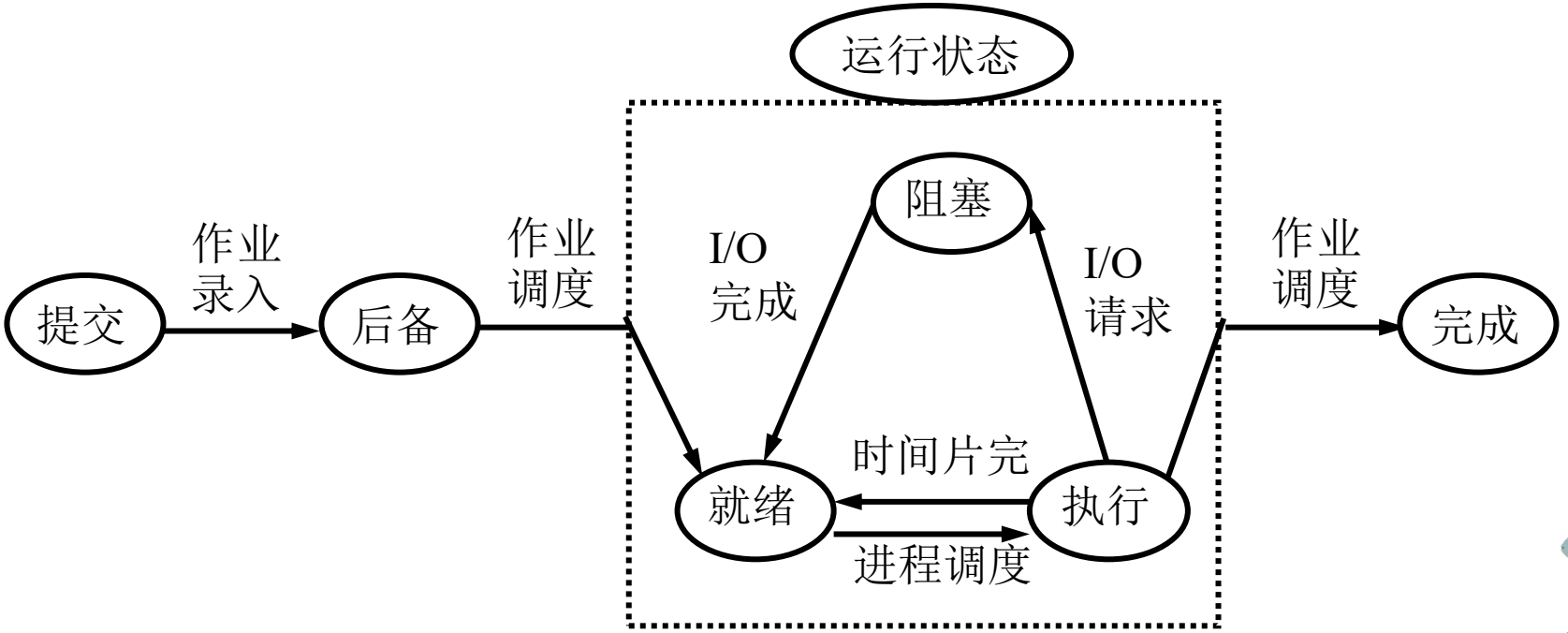

ready queue
CPU
I/O
I/O wait queue
I/O request
time slice expired
child terminates
child termination wait queue
create child process
interrupt occurs
interrupt wait queue
wait for an interrupt

CPU调度

- 长程调度（或作业调度）：从后备队列选择作业装入内存，变为就绪的进程
  - 内存的守门员，控制进程启动，控制多道程序度
  - 注意：只有计算任务才接受作业调度，交互任务必须立刻启动，生来就是进程
  - 多道批处理系统有长程调度，分时系统可能有
- 短程调度（或CPU调度）：从就绪队列选择下一个执行的进程
  - 所有类型的OS（多道批处理、分时）必然有

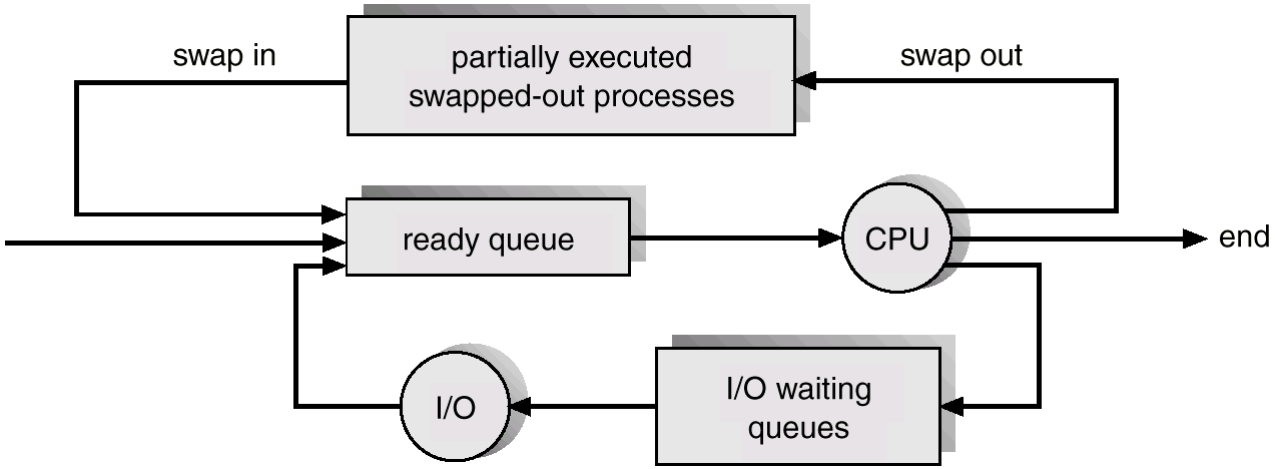
CPU调度

- 作业调度与进程调度的关系
  - 计算任务提交后只是作业，经过作业调度装入内存，才变为进程接受CPU调度，直至计算完成



CPU调度

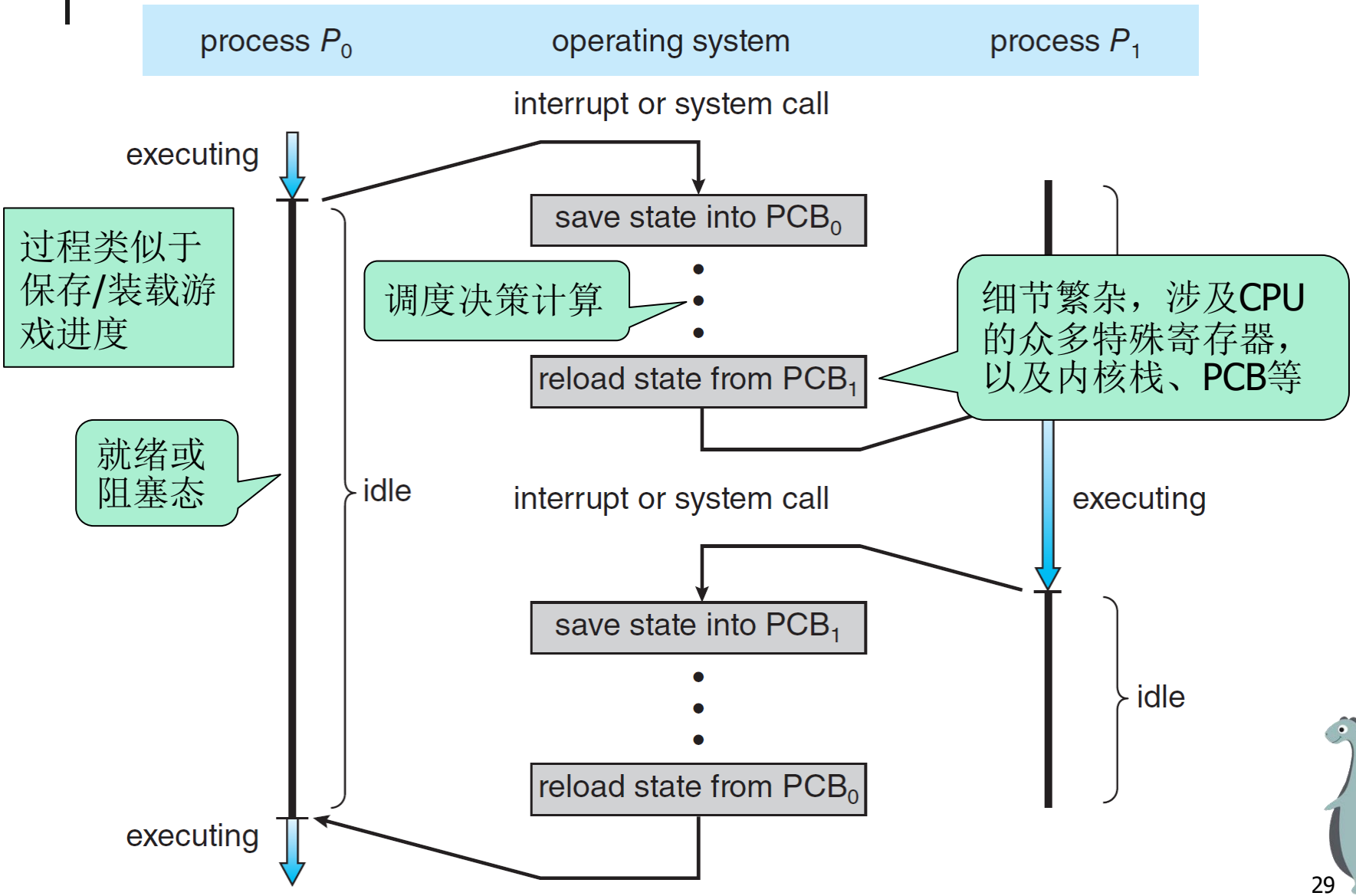
- 进程可以分为
  - I/O型进程：花费I/O 时间多于计算
  - CPU型进程：花费更多时间用于计算
- 有的系统还引入了中程调度
  - 本质上是内存管理手段，将某些进程整体临时换出内存（挂起），被虚拟内存机制淘汰



上下文切换（Context Switch）

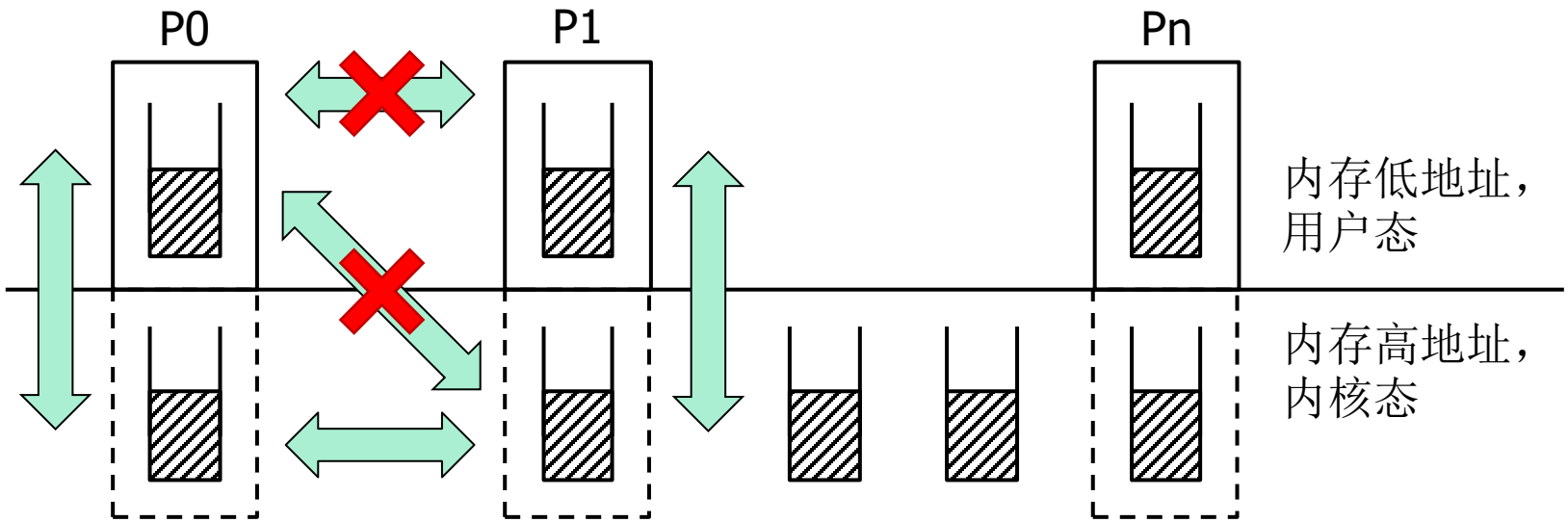
- 将CPU切换到另一个进程需要保存当前进程的状态并恢复另一个进程的状态，这一任务称为上下文切换
  - 进程的上下文保存在PCB中
- 上下文切换的时间开销较重
  - 在切换时，系统没有做有用的工作
  - 时间取决于硬件的支持

上下文切换（Context Switch）



两种“切换”的讨论

- 重要常识：OS在内核中为每个进程准备了一个栈，称为该进程的内核栈
- 两种切换都导致程序的跳转
  - 纵向：内核态与用户态的切换
  - 横向：进程间的切换



两种“切换”的讨论

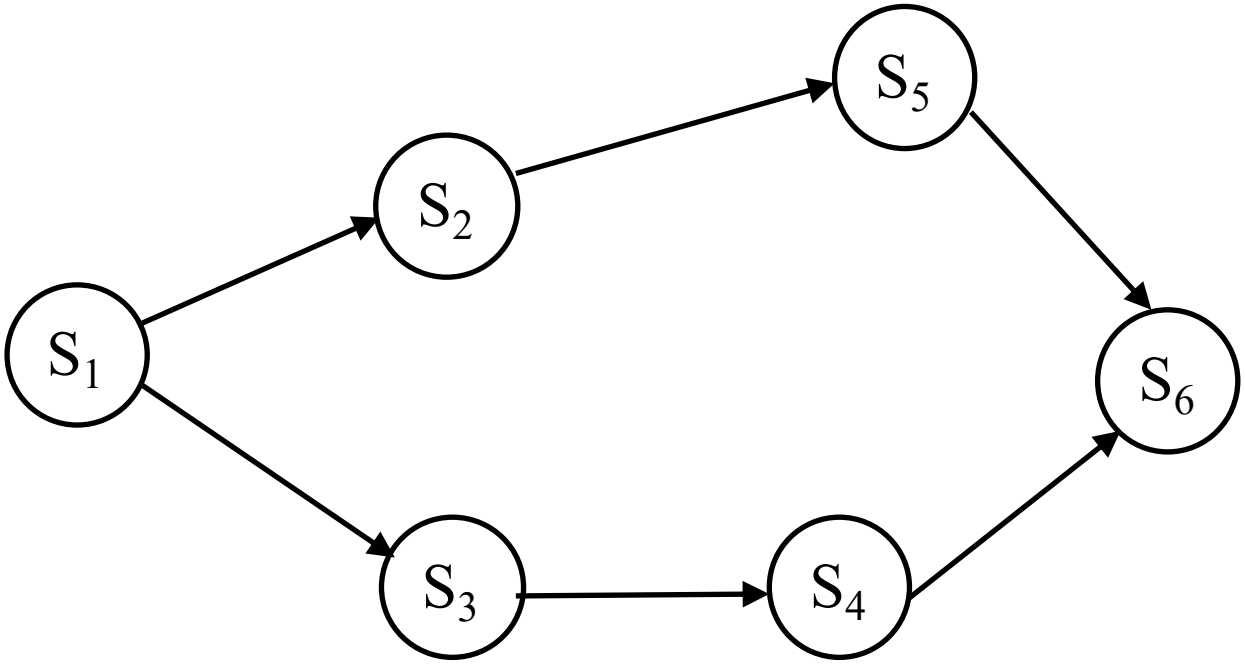
- 用户态程序失去CPU，只有两种原因：
  - 1. 还能继续执行，但发生中断，CPU被OS抢走，转就绪态
    - 中断那一刻，纵向切换至内核态
  - 2. 发起系统调用，OS无法立即完成，转阻塞态
    - 系统调用那一刻，纵向切换至内核态
  - 内核处理完中断或系统调用，若决定切换进程，则发生横向切换
- 因此，切换到内核态是进程切换的前提，用户态程序不可能横向切换

第3章 进程

- 本章内容
- 1. 进程的概念
- 2. 进程调度
- 3. 进程操作
- 4. 进程间通信

补充：前趋图

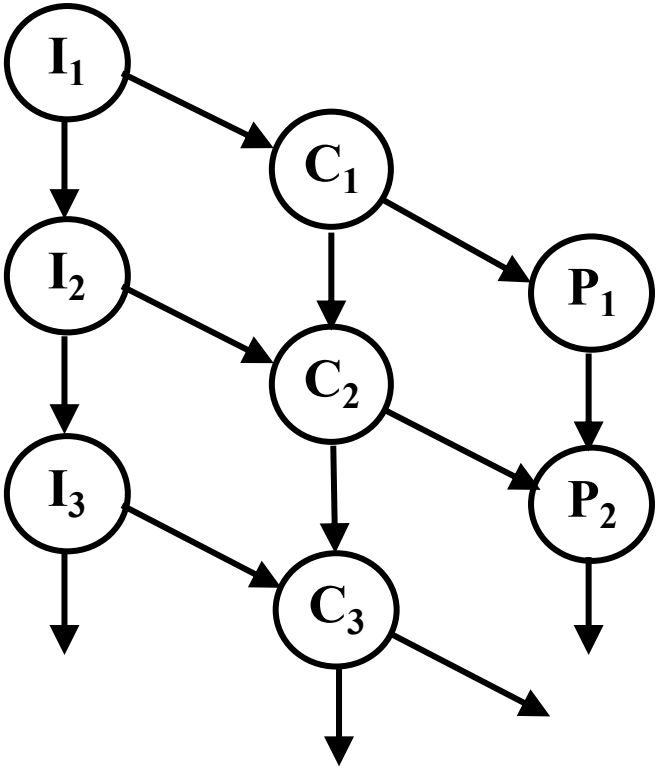
- 前趋图是一个有向无循环图
  - 用于描述程序、程序段或语句执行的先后次序
  - A→B：A必须在B开始执行之前完成



补充：前趋图

- 并发执行的代码往往存在制约，须设法控制执行速度，不可放任

本例中，进程1、2、3并发执行
对每个进程而言，其输入、计算和输出这三个操作必须顺序执行，且相互受制约



补充：并发错误例

- 程序并发执行可能出现与时间有关的错误
- 例

```
进程1：  r1=x;        进程2：  r2=x;
         r1++;                 r2++;
         x=r1;                 x=r2;
```

- 设在两进程运行之前，x的值为0。则两进程运行结束后，x值可能为：

1　　2

并发一旦涉及共享，须格外小心

补充：并发错误例

- 读集：程序段S执行期间要引用的变量集合，记为$R(S) = \{a_1, \ldots, a_m\}$
- 写集：程序段S执行期间要改变的变量集合，记为$W(S) = \{b_1, \ldots, b_n\}$
- Bernstein条件能保证两个程序段P、Q并发执行而不会产生与时间有关的错误
  - $R(P) \cap W(Q) = \{\}$
  - $R(Q) \cap W(P) = \{\}$
  - $W(P) \cap W(Q) = \{\}$

然而，现实中Bernstein条件往往不满足，
例如：后台进程负责计算，前台UI进程展示结果

进程操作与原语

- 常见的功能：创建、撤消、阻塞、唤醒等
- 这些功能一般由操作系统内核原语来实现
  - 原语是一段程序，在执行期间不可分割
    - 不可分割：即一股脑执行完，中间不发生切换
  - 原语用途：访问共享的、敏感的资源
  - 为什么进程操作要使用原语？
    - 涉及全局PCB链表，若操作中途打断，并切换执行其它进程操作，可能造成链表错乱

进程操作与原语

- 原语实现：开头关中断，完成后重开中断
  - 关中断是特殊指令，使CPU不理会外部中断
    - 注意1：中断仍然会产生，只是CPU不理会
    - 注意2：仅限外部中断（包括定时器）
    - 不受中断打扰，CPU自然不会被切换
    - 思考：开关中断是特权指令还是普通指令？原语是内核程序还是用户态程序？
  - 问题：仅适用单CPU，多CPU时，关中断仅影响当前CPU，其它CPU仍可同时执行危险操作

问题3.2：多CPU怎么办？

进程创建

- 通过创建进程系统调用可以创建新进程，创建方称为父进程，被创建方称为子进程
  - 新进程可以再创建新进程，从而形成进程树
  - 进程树又称进程图或进程家族树
- 理解
  - 进程的创建需要OS执行，但多数进程都是在现有进程的要求下创建的
  - Linux的bash进程创建vim进程
  - Windows的Explorer创建QQ进程

进程创建

- 资源共享
  - 父子进程共享所有资源
  - 子进程共享父进程资源的子集
  - 父子进程无资源共享
- 执行方式
  - 父进程和子进程并发执行
  - 父进程等待，直到子进程终止

进程创建

- 地址空间
  - 子进程是父进程的复制品
  - 子进程装入一个新程序
- 导致进程创建的原因
  - 用户登录：用户登录后，若合法则为用户创建一个进程
  - 作业调度：为调度到的作业分配资源并创建进程
  - OS服务：创建服务进程
  - 应用需要：应用程序根据需要创建子进程

进程创建

- 创建原语
  - 向系统申请一个空闲PCB
  - 为新进程分配资源。如分配内存空间
  - 初始化新进程的PCB。在其PCB中填入进程名、家族信息、程序和数据地址、进程优先级、资源清单及进程状态等
  - 将新进程的PCB插入就绪队列

进程终止

- 当进程使用系统调用请求操作系统删除自身时，进程终止，也称为进程撤消
  - 所有进程资源由操作系统回收
- 引起进程撤销的原因
  - 正常结束
  - 异常结束：超时、内存不足、地址越界、算术错、I/O故障、非法指令等
  - 外界干预：包括操作员或系统干预，父进程请求

进程终止

- 父进程终止子进程的原因
  - 子进程超量使用分配的资源
  - 赋予子进程的任务不再需要
  - 父进程退出
    - 有些系统中，若父进程终止，不允许子进程继续
    - 这种现象，称为级联终止
- 撤消的两种策略
  - 撤消指定标识符的进程
  - 撤消指定进程及其所有子孙进程

进程终止

- 撤消原语
  - 从系统的PCB表中找到被撤消进程的PCB
  - 检查被撤消进程的状态是否为执行状态，若是则立即停止该进程的执行，设置重新调度标志
  - 检查被撤消进程是否有子孙进程，若有子孙进程还应撤消该进程的子孙进程
  - 回收该进程占有的全部资源并回收其PCB

进程阻塞与唤醒

- 引起进程阻塞及唤醒的事件
  - 请求系统服务
    - 如请求打印但打印机忙则阻塞；打印机空闲时应唤醒
  - 启动某种操作并等待操作完成
    - 如启动I/O操作，进程阻塞；I/O完成则唤醒进程
  - 等待合作进程的协同配合
    - 如计算进程未算完则打印进程阻塞；算完后应唤醒
  - 系统进程无新工作可做
    - 如没有信息可供发送，则发送请求阻塞；收到新的发送请求时，应唤醒

进程阻塞与唤醒

- 进程由执行变为阻塞状态，是其主动导致
  - 该进程发起系统调用，进而触发阻塞原语
- 阻塞原语
  - 停止当前进程的执行
  - 保存该进程的CPU现场信息
  - 将进程状态改为阻塞，并插入到相应事件的等待队列中
  - 转进程调度程序，从就绪队列中选择一个新的进程投入运行

进程阻塞与唤醒

- 当进程等待的事件发生时，由发现者将其唤醒
  - 发现者可能是进程，如计算完成
  - 也可能是系统，如收到设备中断
- 唤醒原语
  - 将被唤醒进程从相应的等待队列中移出
  - 将进程状态改为就绪，并将该进程插入就绪队列
  - 转进程调度或返回

进程挂起与激活

- 挂起原语和激活都有多种实现方式，如
  - 把请求挂起的进程自身挂起
  - 挂起具有指定标识符的进程
  - 把某进程及其子孙进程挂起
  - 激活一个具有指定标识名的进程
  - 激活某进程及其子孙进程

进程挂起与激活

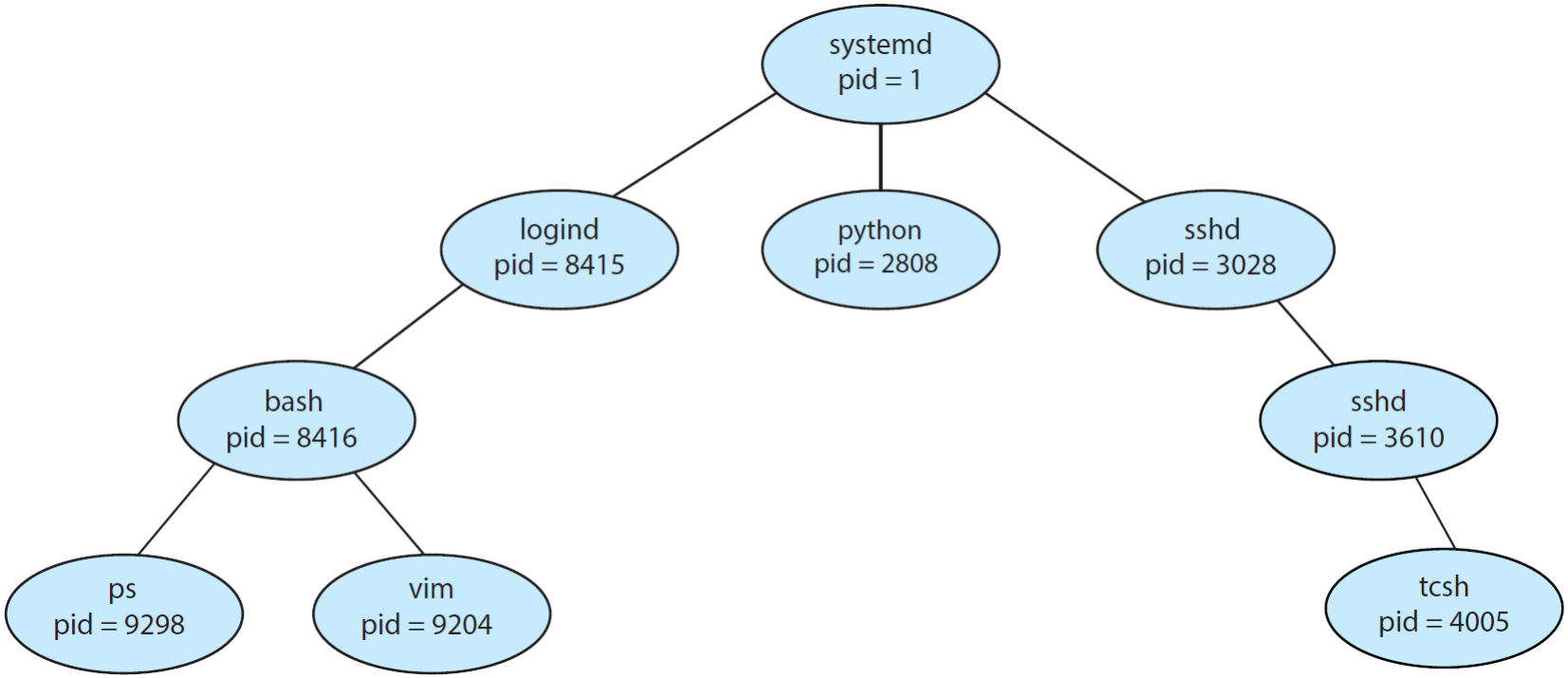
- 挂起原语
  - 到PCB表中查找该进程的PCB
  - 检查该进程的状态，若为执行则停止执行并保护CPU现场信息，将该进程状态改为挂起就绪
  - 若为活动阻塞，则将该进程状态改为挂起阻塞
  - 若为活动就绪，则将该进程状态改为挂起就绪
  - 若进程挂起前为执行状态，则转进程调度，从就绪队列中选择一个进程投入运行

进程挂起与激活

- 激活原语
  - 到PCB表中查找该进程的PCB
  - 检查该进程的状态。若状态为挂起阻塞，则将该进程状态改为活动阻塞
  - 若状态为挂起就绪，则将该进程状态改为活动就绪
  - 若进程激活后为活动就绪状态，可能需要转进程调度

Linux案例

- 进程树


systemd
pid = 1
logind
pid = 8415
python
pid = 2808
sshd
pid = 3028
bash
pid = 8416
sshd
pid = 3610
ps
pid = 9298
vim
pid = 9204
tcsh
pid = 4005

Linux案例

- 进程相关命令
  - ps -ef或ps -aux，配合grep
  - kill -9 pid

```
[root@VM-0-5-centos ~]# ps -ef
UID        PID  PPID  C STIME TTY          TIME CMD
[root@VM-0-5-centos ~]# ps -ef | grep python
root       735     1  0  2024 ?        00:09:16 /usr/bin/python2 -Es /usr/sbin/firewalld --nofork --nopid
root      1177     1  0  2024 ?        00:53:47 /usr/bin/python2 -Es /usr/sbin/tuned -l -P
root      9611  4578  0 17:47 pts/2    00:00:00 grep --color=auto python
root         7     2  0  2024 ?        00:15:43 [migration/0]
root         8     2  0  2024 ?        00:00:00 [rcu_bh]
root         9     2  0  2024 ?        04:26:57 [rcu_sched]
root        10     2  0  2024 ?        00:00:00 [lru-add-drain]
root        11     2  0  2024 ?        00:02:02 [watchdog/0]
root        12     2  0  2024 ?        00:01:35 [watchdog/1]
root        13     2  0  2024 ?        00:15:28 [migration/1]
root        14     2  0  2024 ?        00:22:24 [ksoftirqd/1]
root        16     2  0  2024 ?        00:00:00 [kworker/1:0H]
root        18     2  0  2024 ?        00:00:00 [kdevtmpfs]
root        19     2  0  2024 ?        00:00:00 [netns]
root        20     2  0  2024 ?        00:00:23 [khungtaskd]
root        21     2  0  2024 ?        00:00:00 [writeback]
root        22     2  0  2024 ?        00:00:00 [kintegrityd]
root        23     2  0  2024 ?        00:00:00 [bioset]
root        24     2  0  2024 ?        00:00:00 [bioset]
root        25     2  0  2024 ?        00:00:00 [bioset]
root        26     2  0  2024 ?        00:00:00 [kblockd]
root        27     2  0  2024 ?        00:00:00 [md]
root        28     2  0  2024 ?        00:00:00 [edac-poller]
root        29     2  0  2024 ?        00:00:00 [watchdogd]
root        35     2  0  2024 ?        00:02:18 [kswapd0]
root        36     2  0  2024 ?        00:00:00 [ksmd]
```

Linux案例

- 进程相关命令
  - top
  - pstree

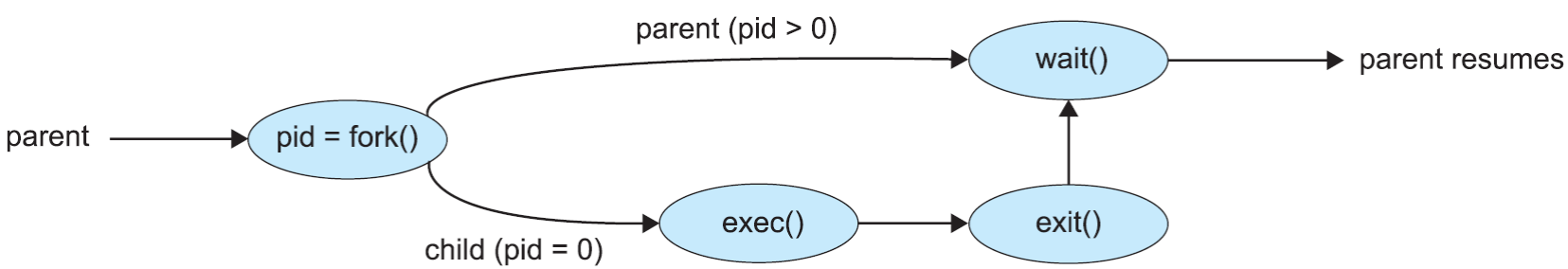
```
[root@VM-0-5-centos ~]# pstree
system
```

```
top - 17:54:36 up 450 days, 16:52,  1 user,  load average: 0.28, 0.23, 0.21
Tasks: 244 total,   2 running, 242 sleeping,   0 stopped,   0 zombie
%Cpu(s):  2.9 us,  2.3 sy,  0.0 ni, 94.8 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
KiB Mem :  8008760 total,   455780 free,  3614604 used,  3938376 buff/cache
KiB Swap:  8257532 total,  6756372 free,  1501160 used.  3086544 avail Mem

  PID USER      PR  NI    VIRT    RES    SHR S  %CPU %MEM     TIME+ COMMAND
31757 root      20   0 4569460 516396   6180 R  13.9  6.4 171:33.76 python3
 9275 root      20   0   32.6g   2.1g 483968 S   5.3 27.4   2161:23 tidb-server
  807 root      20   0  414528   2000   1356 S   0.7  0.0 933:54.38 vmtoolsd
    9 root      20   0       0      0      0 S   0.3  0.0 688:00.12 rcu_sched
 1202 root      20   0 1766612  12096   1564 S   0.3  0.2 487:12.66 containerd
 1756 mysql     20   0 2251540    484      0 S   0.3  0.0   5397:25 mysqld
 8302 root      20   0       0      0      0 S   0.3  0.0   0:00.45 kworker/2:2
 8354 root      20   0  162240   2416   1588 R   0.3  0.0   0:00.10 top
 9036 500       20   0 1787936   6444   5544 S   0.3  0.1  22:33.94 oracle
 9450 root      20   0 5942200 938244   6344 S   0.3 11.7  98:36.69 java
32294 polkitd   20   0   52956    796    368 S   0.3  0.0 139:58.73 redis-server
32642 root      20   0   52956   2564    848 S   0.3  0.0 140:02.22 redis-server
    1 root      20   0  128168   4092   2456 S   0.0  0.1   7:31.02 systemd
    2 root      20   0       0      0      0 S   0.0  0.0   0:22.74 kthreadd
    4 root       0 -20       0      0      0 S   0.0  0.0   0:00.00 kworker/0:0H
    6 root      20   0       0      0      0 S   0.0  0.0   5:39.57 ksoftirqd/0
    7 root      rt   0       0      0      0 S   0.0  0.0   0:15.90 migration/0
    8 root      20   0       0      0      0 S   0.0  0.0   0:00.00 rcu_bh
   10 root       0 -20       0      0      0 S   0.0  0.0   0:00.00 lru-add-drain
```

Linux案例

- 相关API函数
  - fork()
  - wait()和waitpid()
  - exec()
  - 自学：void exit(int status);
  - 自学：int kill(pid_t pid, int sig);



Linux案例

- fork()
  - 程序是事先编好的、静态的
  - 进程是从main()开始爬行的虫子
    - 虫子有自己的PCB、内存数据、打开的文件等
  - fork()使虫子被克隆
    - PCB、内存数据等都被复制一份
    - 两只虫子带着相同的状态，面对相同的程序继续执行

问题3.3：fork()把全部数据都复制一份，岂不是需要很久，容易造成卡顿？

```
...
int pid = fork();      //此处之后有了两只虫，都要继续执行
...
```

Linux案例

- fork()
  - 父进程感到自己刚调用完fork()，子进程也一样
  - 区别：返回值
    - 父进程拿到的是子进程的PID
    - 子进程拿到0
  - 父子进程执行同一份程序，配合完成任务

```
...
int pid = fork();      //此处之后有了两只虫，都要继续执行
if(pid == 0) {
        ...     //子进程
} else {
        ...     //父进程
}
```

Linux案例

- wait()
  - 子进程结束后不会立即消失，而是变成僵尸进程
    - 会留一些资料（如退出状态）供事后调查
  - wait()或waitpid()，阻塞等待回收僵尸子进程
    - 负责任的父进程应该调用，最后结束

```
...
int pid = fork();      //此处之后有了两只虫，都要继续执行
if(pid == 0) {
        ...     //子进程
} else {
        ...     //父进程
        wait(NULL);
}
```

Linux案例

- wait()
  - 特殊情况
    - 父进程忘记wait()，则留下僵尸进程，占据一些资源
    - 如果父进程先结束，孤儿进程（包括僵尸进程）被init进程领养，但init会周期性清理僵尸
    - 如果init的子进程结束，则init进程自动回收其资源，而不是让它变成僵尸进程

Linux案例

- exec()
  - 有时创建子进程不是为了配合父进程，而是为了让它运行另一个可执行文件
  - 应在子进程路径上调用exec系列函数
    - 执行后，子进程变身，面目全非
    - 但从父进程克隆来的打开的文件仍得保留
  - 并非一个函数，而是以exec开头的函数族
    - 例：int execlp(const char *file, const char *arg, ...);
    - file是可执行文件路径，arg是程序名
    - 后面是传递给程序的参数，最后一个必须是NULL

Linux案例

- exec()

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>
int main() {
        pid_t pid;
        pid = fork();
        if (pid < 0) {
                fprintf(stderr, "Fork Failed");
                return 1;
        } else if (pid == 0) {
                execlp("/bin/ls","ls",NULL);
        } else {
                wait(NULL);
                printf("Child Complete");
        }
        return 0;
}
```

Windows案例

```
#include <stdio.h>
#include <windows.h>
int main() {
        STARTUPINFO si;
        PROCESS_INFORMATION pi;
        ZeroMemory(&si, sizeof(si));
        si.cb = sizeof(si);
        ZeroMemory(&pi, sizeof(pi));
        if (!CreateProcess(NULL,
                "C:\\WINDOWS\\system32\\mspaint.exe",
                NULL, NULL, FALSE, 0, NULL, NULL, &si, &pi))
        {
                fprintf(stderr, "Create Process Failed");
                return -1;
        }
        WaitForSingleObject(pi.hProcess, INFINITE);
        printf("Child Complete");
        CloseHandle(pi.hProcess);
        CloseHandle(pi.hThread);
}
```

第3章 进程

- 本章内容
- 1. 进程的概念
- 2. 进程调度
- 3. 进程操作
- 4. 进程间通信

进程通信的类型

- 协作进程间需要通信机制来交换信息
  - 低级通信
    - 交换的是控制信息，目的是控制进程的执行
    - 进程互斥与同步（后续章节学习）
    - Linux信号机制
  - 高级通信
    - 目的是传送数据，接收方可以看到、处理数据
    - 共享存储器系统（后续章节学习）
    - 消息传递系统
    - 管道通信系统或共享文件系统

Linux信号机制

- 自行搜索和学习
  - 信号机制，SIGINT和进程组
  - kill()，raise()，alarm()，setitimer()
  - signal()，sigaction()
  - 注意：信号机制（signal）与后面要讲的信号量（semaphore）完全无关
- 进一步的学习建议
  - 进程组、会话、控制终端之间的关系
  - 前台进程、后台进程和守护进程之间的差异

Linux消息机制

- 自行搜索和学习
  - 消息队列机制，阻塞与非阻塞模式
  - System V
    - msgget()，msgctl()
    - msgsnd()，msgrev()，msgflg参数与阻塞设定
  - POSIX
    - mq_open()，mq_close()，mq_unlink()
    - mq_getattr()，mq_setattr()
    - mq_send()，mq_receive()，mq_notify()

Linux管道机制

- 自行搜索和学习
  - 一般的文件读写
    - open()，close()，read()，write()，文件描述符
  - 匿名管道：pipe()
    - 阻塞条件，单方面关闭的后果
  - 命名管道：mkfifo()，unlink()
    - 阻塞条件
- 进一步的学习建议
  - popen()，pclose()

练习题

- 3.1 Using the program shown in below, explain what the output will be at LINE A.

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

int value = 5;

int main()
{
pid_t pid;

   pid = fork();

   if (pid == 0) { /* child process */
      value += 15;
      return 0;
   }
   else if (pid > 0) { /* parent process */
      wait(NULL);
      printf("PARENT: value = %d",value); /* LINE A */
      return 0;
   }
}
```

Figure 3.30 What output will be at Line A?

练习题

- 3.2 Including the initial parent process, how many processes are created by the program shown in Figure 3.31?

```
#include <stdio.h>
#include <unistd.h>

int main()
{
    /* fork a child process */
    fork();

    /* fork another child process */
    fork();

    /* and fork another */
    fork();

    return 0;
}
```

Figure 3.31 How many processes are created?

选择题1

- 对进程的管理和控制使用_____。

  A. 指令　　B. 信号量

  C. 原语　　D. 信箱

- 分配到必要的资源并获得处理机时的进程状态是_____。

  A. 就绪状态　　B.撤消状态

  C. 执行状态　　D.阻塞状态

选择题2

- 下列进程状态变化中，_____变化是不可能发生的。

  A.等待→就绪　　B.等待→运行

  C.运行→等待　　D.运行→就绪

- 当_____时，进程从执行状态转变为就绪状态。

  A.等待的事件发生　　B.时间片到

  C. 等待某一事件　　D.进程被调度程序选中

选择题3

- 下面对进程的描述中，错误的是_____。

  A.进程是有生命期的　B. 进程执行需要处理机

  C.进程是指令的集合　D.进程是动态的概念

- 如果系统中有n个进程，则就绪队列中进程的个数最多为_____。

  A. n　B. 1　C. n-1　D. n+1

选择题4

- 操作系统通过_____对进程进行管理。

  A. JCB　　B. PCB

  C. DCT　　D. CHCT

- 下面所述步骤中，_____不是创建进程所必需的。

  A.建立一个进程控制块

  B.为进程分配内存

  C. 将进程控制块链入就绪队列

  D.由调度程序为进程分配CPU

选择题5

- 下述哪一个选项，体现了原语的主要特点_____。

  A. 并发性　　B. 异步性

  C.不可分割性　　D.共享性

- 下面对父进程和子进程的叙述不正确的是_____。

  A. 撤消父进程之时，可以同时撤消其子进程

  B. 父进程和子进程之间可以并发

  C. 父进程可以等待所有子进程结束后再执行

  D.父进程创建了子进程，因此父进程执行完了子进程才能运行

选择题6

- 下列几种关于进程的叙述中，最不符合操作系统对进程理解的是_____。

A．进程是在多程序并行环境中的完整的程序

B．进程可以由程序，数据和进程控制块描述

C．线程(Thread)是一种特殊的进程

D．进程是程序在一个数据集合上运行的过程，是系统进行资源分配和调度的一个独立单位

选择题7

- 当一个进程处于_____的状态时，称其为等待状态

  A. 它正等待调度

  B. 它正等着协作进程的一个消息

  C. 它正等分给它一个时间片

  D. 它正等进入内存

- 进程从执行状态到阻塞状态可能是由于_____ 。

  A. 进程调度程序的调度

  B. 现运行进程的时间片用完

  C. 现运行进程执行了P操作

  D. 现运行进程执行了V操作

选择题8

- 一个进程被唤醒意味着_____。

  A. 该进程重新占有了CPU

  B. 进程状态变为就绪

  C. 它的优先权变为最大

  D. 其PCB移至就绪队列的队首

- 一个进程基本状态可以从其他两种基本状态转变过来，这个基本状态是_____。

  A. 执行状态　　B. 阻塞状态

  C. 就绪状态　　D. 撤销状态

选择题9

- 设系统中有n（n>2）个进程，且当前不在执行进程调度程序，试考虑下述4种情况：

  1. 有1个运行进程，n-1个就绪进程，没有进程处于等待状态。
  2. 有1个运行进程，没有就绪进程，n-1进程处于等待状态。
  3. 有1个运行进程，有1个就绪进程，n-2进程处于等待状态。
  4. 没有运行进程，有2个就绪进程，n个进程处于等待状态

  上述情况中，不可能发生的情况是_____。

选择题10

- 下面关于进程的叙述中，不正确的有 _____ 条。

  ① 进程申请CPU得不到满足时，其状态变为等待状态。

  ② 在单CPU系统中，任一时刻都有一个进程处于运行状态。

  ③ 优先级是进行进程调度的重要依据，一旦确定不能改变。

  ④ 进程获得处理机而运行是通过调度而实现的。

考研题1

- 下列选项中，导致创建新进程的操作是____。

Ⅰ 用户登录成功　　Ⅱ 设备分配

Ⅲ 启动程序执行

A.仅Ⅰ和Ⅱ　　B.仅Ⅱ和Ⅲ

C.仅Ⅰ和Ⅲ　　D.Ⅰ、Ⅱ、Ⅲ

- 下列选项中，在用户态执行的是____。

A、命令解释程序　B、缺页处理程序

C、进程调度程序　D、时钟中断处理程序

考研题2

- 下列选项中，不可能在用户态发生的事件是（）。

A. 系统调用　　B. 外部中断

C. 进程切换　　D. 缺页